Typst Cheat Sheet

Command	Description
`typst compile file.typ`	one-shot compile to PDF/PNG/SVG/HTML
`typst watch file.typ`	incremental recompile on save
`typst fonts`	list discovered system fonts
`typst init @preview/pkg-name`	scaffold project from template
`typst compile --font-path inc/fonts file.typ`	add custom font directories

Concept	Syntax / Example	Output / Notes
Content block	`[Hello *world*]`	Hello **world** fundamental immutable type; evaluates markup
Code block	`#{ let x = 5 x * 2 }`	10 evaluates script, returns last value
Code in markup	`Result: #(30 + 12)`	Result: 42
Math mode	`$x^2 + y^2 = z^2$` `$ x^2 + y^2 = z^2 $`	$x^2 + y^2 = z^2$ (inline) $$x^2 + y^2 = z^2$$ (display) spaces inside trigger display mode

Concept	Code	Output / Notes
Binding	`#let x = 5`	`5` — immutable by default
Function	`#let add(x, y) = x + y`	`#let add(x, y) = x + y`
Lambda	`#(x, y) => x + y`	anonymous function
Conditional	`#if x > 5 [big] else [small]`	returns content
For loop	`#for name in list [Name: #name]`	yields content for each item
Dict loop	`#for (k, v) in dict [Key: #k]`	destructure key-value pairs
While	`#while x < 10 { x += 1 }`	condition-controlled loop
Context	`#context [Page #counter(page).get()]`	layout-aware evaluation

Pattern	Description
`#set heading(numbering: "1.")`	Set rule — customize element parameters. Always overridable.
`#show heading: set text(fill: navy)`	Show-set rule — change element properties. Composable (can be overridden later).
`#show heading: it => { v(0.5em); it; v(0.3em) }`	Show-transform — full control over rendering. NOT overridable later. Receives element, returns content.
`#show heading.where(level: 1): set text(16pt)`	Target by condition: `.where(level: N)`, `.where(outline: true)`, or by label: `#show <label>: set ...`
`#show: doc => { set text(font: "Lib Serif", 11pt) doc }`	Global show rule — wrap entire document body. Standard template pattern.

Pattern	Code	Notes
Template	`#show: doc => template(doc, title: [T], author: [A])`	wrap body in layout function; define `template()` in separate file
Draft mode	`#let draft = true` / `#show: doc => { set text(fill: red); doc }`	conditional formatting; toggle before submission
Custom heading	`#show heading: it => { v(0.5em, weak: true); set text(weight: "bold"); it; line(length: 100%, stroke: 0.3pt); v(0.2em) }`	spacing + rule under heading; `weak: true` collapses at page break
Hide empty	`#show heading: it => if it.body == [] { hide(it) } else { it }`	suppress empty headings from outline/TOC
Page numbers	`#set page(numbering: context [(#counter(page).display())])`	custom page number format; needs `#context` for counter access
Multi-column	`#set page(columns: 2); #set columns(gutter: 0.5cm); #colbreak()`	2-column layout; gutter = space between columns
Text highlight	`#show "TODO": it => highlight[TODO]`	auto-highlight keywords via show rule on text

Function	Example / Output
`#v(12pt, weak: true)` `#h(1em)`	vertical / horizontal space. `weak: true` = collapses at page break.
`#pad(x: 1em, y: 0.5em, [content])`	Pad content on all sides. `rest:` for uniform padding.
`#align(center, [text])` `#align(center + horizon, [x])`	Align: left, center, right, top, bottom, horizon. Combine with +.
`#place(top + right, float: true,` `  dx: 5pt, [x])`	Absolute placement. `scope: "parent"` for page-level. `float: true` avoids layout impact.

`#block(width: 3cm, fill: aqua.lighten(80%),`
`  stroke: 0.5pt, radius: 4pt)[x]`
`#box(fill: gray.lighten(90%), [inline])`
	x
	inline
	`block`: sized container (block-level). `box`: inline container (no break).

`#stack(dir: ltr, spacing: 4pt,` `  [A], [B], [C])`	A B C dir: ltr, rtl, ttb. `spacing:` gap between items.

`#grid(columns: 3, gutter: 4pt,`
`  [a], [b], [c],`
`  [d], [e], [f])`
	a b c
	d e f
	2D grid layout. `rows:` optional.

`#hide([hidden])`
`#move(dx: 2pt, dy: 0, [offset])`
`#rotate(45deg, [tilted])`
`#scale(x: 150%, [big])`
	`hide`: remove from output. `move`: offset from flow position. `rotate`/`scale`: transform content.

)

Math	Code
Fraction	`$frac(a, b)$` or `$a/b$`
Sup/subscript	`$x^2$, $x_1$, $x_(n+1)$`
Root	`$sqrt(x)$, $root(3, x)$`
Sum / integral	`$sum_(i=0)^n i^2$, $integral_a^b f(x) dif x$`
Matrix	`$mat(1, 2; 3, 4)$`
Piecewise	`$cases(0 "if" x<0, 1 "else")$`
Aligned	`$ x &= 2 \ y &= 3 $`
Accents	`$tilde(x)$, $hat(x)$, $dot(x)$, $bar(x)$`
Sets / symbols	`$x in RR$, $emptyset$, $infty$, $nabla$, $partial$`

Math	Code
Text in math	`$bold(x)$, $italic(x)$, $a "is natural"$`
Custom op	`$op("cis", x)$`

Concept	Syntax / Notes

Import	`#import "lib.typ": f1, f2`
	`#import "lib.typ" as lib`
	`#import "@preview/pkg:1.0": *`
	`import` returns a **module** (vars & functions).

Include	`#include "chapter.typ"`
	`#include "data.json"`
	`include` returns **content** (in-place). Parses JSON/YAML.

Labels & refs	`<my-label>` `// place label`
	`@my-label` `// reference`
	`@my-label[Custom]` `// custom text`

Figures	`#figure(`
	`  image("img.png", width: 80%),`
	`  caption: [Caption],`
	`) <fig-label>`

Tables	`#table(`
	`  columns: (auto, 1fr, 30%),`
	`  stroke: 0.5pt, inset: 6pt,`
	`  align: (left, center, right),`
	`  fill: (x, y) => if y == 0 { gray },`
	`  table.header([Name], [Desc]),`
	`  table.hline(),`
	`  [a], [b],`
	`)`
	`table.cell(colspan: 2, rowspan: 1, [x])` for merged cells.

Text styling	`#strong[bold] #emph[italic] #highlight[yellow]`
	`#strike[gone] #underline[line] #overline[bar]`
	`#sub[sub] #super[super] #smallcaps[SC]`
	`#text(fill: blue, size: 1.2em, weight: "bold")[text]`

Counters	`#counter("my-ctr").step()`
	`#counter("my-ctr").display()`
	`#context counter("my-ctr").get()`
	Built-in: `counter(page)`, `counter(heading)`, `counter(figure)`.

Bibliography	`#bibliography("refs.bib", style: "apa")`
	`@cite-key` or `@cite-key[page 42]`
	Styles: apa, mla, chicago, ieee, nature.